👍

TASK 1: Student Performance Management System (Streamlit + MySQL)

Project Description

Build a **Streamlit web app** where users can **add, view, update, and analyze student performance data** stored in a database.

Functional Requirements

- Add student details (Name, Age, Subject, Marks)
- Store data in **MySQL**
- View all students in table format
- Update marks
- Delete student record
- Show **Pass/Fail status**
- Show **average marks per subject**

- Calculate:
  - Average marks
  - Pass percentage
  - Top scorer
- Visualize:
  - Bar chart of subject vs average marks
  - Pie chart of pass/fail ratio

Tech Stack

- Frontend: **Streamlit**
- Backend: **Python**
- Database: **MySQL**
- Libraries: `pandas`, `mysql-connector-python`, `matplotlib`

Database Table Example

```
students(id, name, age, subject, marks)
```

GitHub Push Instructions

- README.md (project explanation)
- app.py
- db.sql (table creation)
- requirements.txt
- Screenshots folder

TASK 2: Student Attendance & Marks Portal

Description

Build a **Streamlit web application** to manage **student attendance and marks** using a database.

Functional Requirements

- Add student details (Roll No, Name, Class)

- Mark **daily attendance (Present/Absent)**
- Add subject-wise marks
- View attendance history
- Calculate total attendance %
- Show pass/fail status

Streamlit Concepts (Apart from DB)

- `st.form()` – group inputs
- `st.selectbox()` – select class/subject
- `st.radio()` – attendance status
- `st.success()`, `st.error()` – feedback
- `st.session_state` – login/session handling
- `st.dataframe()` – display records

Database Tables

```
students(id, roll_no, name, class)
attendance(id, student_id, date, status)
marks(id, student_id, subject, marks)
```

GitHub Must Contain

- app.py
- database.sql
- requirements.txt
- README.md
- screenshots/

TASK 3: Online Complaint Management System

Description

Create a **complaint registration system** where users submit issues and admin tracks status.

Functional Requirements

- User submits complaint (Name, Email, Category, Description)
- Auto-generate complaint ID
- Store complaints in database
- Admin can:
    - View all complaints
    - Update complaint status (Open / In Progress / Closed)
- Search complaint by ID

Streamlit Concepts (Apart from DB)

- `st.text_area()` – complaint description
- `st.selectbox()` – complaint category
- `st.expander()` – hide/show complaint details
- `st.sidebar()` – admin menu
- `st.button()` – actions
- Input validation

Database Table

```
complaints(
  id,
  name,
  email,
  category,
  description,
  status,
  created_at
)
```

GitHub Must Contain

- app.py
- admin.py
- schema.sql
- README.md (with screenshots)

TASK 4: Inventory & Billing Management App

Description

Develop a **shop inventory and billing system** using Streamlit and database.

Functional Requirements

- Add products (Name, Price, Stock)
- Update stock after purchase
- Generate bill (Product, Quantity, Total)
- Store bills in database
- View daily sales summary

Streamlit Concepts (Apart from DB)

- `st.number_input()` – quantity, price
- `st.columns()` – billing layout
- `st.metric()` – total sales
- `st.download_button()` – download bill
- `st.session_state` – cart handling

Database Tables

```
products(id, name, price, stock)
bills(id, bill_date, total_amount)
bill_items(id, bill_id, product_id, quantity)
```

GitHub Must Contain

- app.py
- billing.py
- database.sql
- README.md
- bill_sample.png